

Q1. Define Agile Manifesto and list its four core values.

The **Agile Manifesto** is a set of principles introduced in **2001** to improve software development. It focuses on teamwork, customer satisfaction, flexibility, and delivering working software quickly. Agile helps teams respond to changing requirements and continuously improve their products. It promotes collaboration, frequent feedback, and faster delivery of high-quality software.

Four Core Values of the Agile Manifesto:

1. **Individuals and interactions** over processes and tools.
2. **Working software** over comprehensive documentation.
3. **Customer collaboration** over contract negotiation.
4. **Responding to change** over following a fixed plan.

Advantages of Agile:

- Faster software delivery.
- Better teamwork and communication.
- Easy to handle changing requirements.
- Higher customer satisfaction.

Q2. What are the Three Ways of DevOps? Explain each briefly.

The **Three Ways of DevOps** are the basic principles that help improve software development and operations. They focus on better workflow, communication, and continuous improvement.

1. Flow

Flow focuses on moving work smoothly from development to testing and deployment. It reduces delays and helps deliver software faster.

2. Feedback

Feedback ensures that developers quickly receive information about bugs and errors. This helps fix problems early and improve software quality.

3. Continuous Learning and Improvement

This principle encourages teams to learn from mistakes, automate repetitive tasks, and continuously improve their processes.

Advantages:

- Faster software delivery.
- Better collaboration.
- Improved software quality.
- Early bug detection.

Q3. List the roles involved in the Scrum framework.

Scrum is an Agile framework used to manage software development projects. It has **three main roles**.

1. Product Owner

- Defines product requirements.
- Manages the Product Backlog.
- Prioritizes work based on customer needs.

2. Scrum Master

- Ensures Scrum practices are followed.
- Removes obstacles faced by the team.
- Guides and supports the Scrum team.

3. Development Team

- Develops, tests, and delivers the software.
- Works together to complete Sprint goals.
- Delivers a working product after each Sprint.

Importance of Scrum Roles:

- Better communication.
- Clear responsibilities.
- Improved teamwork.
- Faster project completion.

Q4. What is Version Control? State its advantages.

Version Control is a system that records and manages changes made to source code or project files over time. It allows developers to track changes, collaborate with team members, and restore previous versions if needed. **Git** is the most commonly used Version Control System.

Advantages of Version Control:

- Tracks all code changes.
- Supports teamwork and collaboration.
- Prevents file overwriting.
- Makes it easy to restore previous versions.
- Helps identify and fix errors.

- Maintains complete project history.

Examples:

- Git
- SVN (Subversion)
- Mercurial

Q5. What is Continuous Integration (CI)? Name any two CI tools.

Continuous Integration (CI) is a DevOps practice where developers frequently merge their code into a shared repository. Every code change is automatically built and tested to detect errors early. This improves software quality and speeds up development.

Benefits of Continuous Integration:

- Detects bugs early.
- Improves code quality.
- Reduces integration problems.
- Saves development time.
- Supports faster software delivery.

Any Two CI Tools:

1. Jenkins
2. GitHub Actions

Other CI Tools:

- GitLab CI/CD
- CircleCI
- Travis CI

Q6. Explain the relationship between Agile methodology and DevOps.

Agile and **DevOps** are closely related approaches used in software development. Agile focuses on developing software quickly through teamwork, short development cycles, and regular customer feedback. DevOps extends Agile by improving collaboration between the development and operations teams to automate testing, deployment, and maintenance.

Agile helps teams build software faster, while DevOps ensures that the software is delivered and maintained efficiently. Both approaches aim to improve software quality, reduce development time, and increase customer satisfaction.

Relationship between Agile and DevOps:

- Both encourage teamwork and collaboration.
- Both support continuous improvement.
- Agile focuses on development, while DevOps focuses on development and operations.
- Both help deliver software faster and with fewer errors.

Q7. Differentiate between Scrum and Kanban with suitable examples.

Scrum and **Kanban** are two popular Agile frameworks used to manage projects. Although both improve productivity, they have different working methods.

Scrum

Work is divided into fixed Sprints.

Team members have defined roles.

Sprint planning meetings are conducted.

Changes are not allowed during a Sprint. Changes can be made at any time.

Kanban

Work flows continuously.

No fixed roles are required.

Example:

- **Scrum:** Developing a mobile banking application in 2-week Sprints.
- **Kanban:** Managing customer support tickets for an e-commerce website.

Q8. Explain the purpose of User Stories and Story Points in Agile development.

A **User Story** is a simple description of a feature from the user's point of view. It explains what the user wants and why it is needed. User Stories help developers understand customer requirements clearly.

Story Points are used to estimate the effort required to complete a User Story. They measure complexity, risk, and time instead of actual hours. Agile teams often use the Fibonacci sequence (1, 2, 3, 5, 8, 13) for estimation.

Benefits:

- Better understanding of customer needs.
- Improved Sprint planning.
- Easy estimation of work.
- Better teamwork and productivity.

Q9. Describe the workflow of Git branching and merging.

Git branching allows developers to work on new features without affecting the main project. Each developer creates a separate branch, makes changes, and tests the code. After the work is completed, the branch is merged into the main branch.

Git Branching Workflow:

1. Create a new branch.
2. Develop the feature in that branch.
3. Commit the changes.
4. Create a Pull Request.
5. Review the code.
6. Merge the branch into the main branch.
7. Delete the completed branch.

Advantages:

- Supports teamwork.
- Prevents code conflicts.
- Keeps the main branch stable.
- Makes code review easier.

Q10. Explain how Continuous Integration improves software quality.

Continuous Integration (CI) is a process where developers regularly merge their code into a shared repository. Every code change is automatically built and tested. This helps identify bugs early and improves software quality.

CI ensures that all code changes work correctly before they are added to the main project. Automated testing reduces human errors and increases development speed.

Benefits of Continuous Integration:

- Detects bugs early.
- Improves code quality.
- Reduces integration issues.
- Saves time and cost.
- Supports faster software delivery.
- Increases team collaboration.

Q11. A software team needs to manage frequent code changes from multiple developers. Demonstrate how Git branching and Pull Requests can be used to avoid conflicts.

When multiple developers work on the same project, **Git branching** and **Pull Requests (PRs)** help manage code changes without creating conflicts. Each developer creates a separate branch to work on a new feature or bug fix. This keeps the main branch safe and stable.

After completing the work, the developer commits the changes and creates a Pull Request. Other team members review the code, suggest improvements if needed, and approve it. Once approved, the branch is merged into the main branch.

Steps:

1. Create a new feature branch.
2. Write and test the code.
3. Commit and push the changes.
4. Create a Pull Request.
5. Review and approve the code.
6. Merge into the main branch.

Benefits:

- Reduces code conflicts.
- Improves collaboration.
- Ensures code quality.
- Keeps the main branch stable

Q12. A startup wants to adopt Agile for product development. Illustrate how Scrum ceremonies can be organized during a two-week sprint.

A **two-week Sprint** is a fixed period in Scrum during which the team develops a set of features. Scrum ceremonies help the team stay organized and complete the work successfully.

Scrum Ceremonies:

1. **Sprint Planning:** The team selects tasks from the Product Backlog and plans the Sprint.
2. **Daily Scrum:** A 15-minute daily meeting to discuss progress, plans, and issues.
3. **Sprint Review:** At the end of the Sprint, the team demonstrates the completed work to stakeholders.
4. **Sprint Retrospective:** The team discusses what went well, what problems occurred, and how to improve in the next Sprint.

Benefits:

- Better communication.

- Regular progress tracking.
- Faster problem solving.
- Continuous improvement.

Q13. Given a project backlog, estimate Story Points for five user stories using Fibonacci estimation.

Story Points are used in Agile to estimate the effort needed to complete a User Story. The **Fibonacci sequence (1, 2, 3, 5, 8, 13)** is commonly used because it reflects increasing complexity.

User Story	Story Points
User Login	2
User Registration	3
Forgot Password	5
Online Payment	8
Admin Dashboard	13

Benefits of Story Points:

- Helps estimate work accurately.
- Improves Sprint Planning.
- Balances team workload.
- Measures task complexity.

Q14. Apply Kanban principles to organize the workflow of an online shopping application development team.

Kanban is an Agile method that helps teams manage work visually using a Kanban board. The work is divided into different stages so everyone can track progress easily.

Kanban Workflow:

- **To Do:** New tasks such as Login, Product Page, Cart, and Payment.
- **In Progress:** Tasks currently being developed.
- **Testing:** Completed tasks are tested for bugs.
- **Done:** Successfully tested tasks are completed.

The team moves each task from one column to the next until it is finished.

Benefits:

- Easy task tracking.

- Better teamwork.
- Faster delivery.
- Reduced work delays.

Q15. Configure a basic CI workflow using GitHub Actions or Jenkins for a Java project that automatically builds and executes unit tests.

A **Continuous Integration (CI)** workflow automatically builds and tests a Java project whenever code is pushed to the repository. This helps detect errors early and improves software quality.

Basic CI Workflow:

1. Developer pushes code to GitHub.
2. GitHub Actions or Jenkins detects the change.
3. The Java project is automatically built.
4. Unit tests are executed.
5. Test results are displayed.
6. If all tests pass, the code is ready for deployment.

Benefits:

- Automatic code building.
- Early bug detection.
- Better software quality.
- Saves development time.
- Faster project delivery.

Q16. Compare Agile and Traditional Waterfall methodologies based on flexibility, customer involvement, and project delivery.

Agile and **Waterfall** are two different software development methodologies. Agile is flexible and allows changes during development, while Waterfall follows a fixed sequence of phases.

Agile

Flexible and accepts changes

Customer is involved throughout the project

Software is delivered in small increments

Waterfall

Fixed plan with limited changes

Customer involvement is mostly at the beginning and end

Software is delivered only after the project is completed

Advantages of Agile:

- Faster delivery.

- Better customer satisfaction.
- Easy to adapt to changes.

Advantages of Waterfall:

- Simple to understand.
- Well-planned documentation.
- Suitable for fixed requirements.

Q17. Analyze the reasons behind the failure of DevOps implementation in an organization.

DevOps implementation may fail if an organization does not follow proper planning and teamwork. Successful DevOps requires collaboration, automation, and continuous improvement.

Reasons for Failure:

- Poor communication between development and operations teams.
- Lack of management support.
- Resistance to change.
- Insufficient training and skills.
- Limited automation.
- Poor planning and unclear goals.

Solutions:

- Improve team collaboration.
- Provide proper training.
- Automate testing and deployment.
- Set clear objectives.
- Encourage continuous learning.

Q18. Compare Git Flow and GitHub Flow branching strategies for enterprise software development.

Git Flow and **GitHub Flow** are popular branching strategies used to manage software development.

Git Flow

Uses multiple branches such as feature, develop, release, and main

Suitable for large projects

GitHub Flow

Uses a single main branch with feature branches

Suitable for small and fast-moving projects

Git Flow

More complex workflow

Best for scheduled releases

GitHub Flow

Simple workflow

Best for continuous deployment

Advantages of Git Flow:

- Better release management.
- Suitable for enterprise projects.

Advantages of GitHub Flow:

- Easy to learn.
- Faster development.
- Supports continuous deployment.

Q19. Analyze the impact of Code Reviews and Protected Branches on software quality.

Code Reviews and **Protected Branches** are important practices that improve software quality and reduce development errors.

Code Reviews allow team members to examine code before it is merged. This helps identify bugs, improve coding standards, and share knowledge.

Protected Branches prevent developers from making direct changes to important branches like the main branch. Changes are allowed only after approval through Pull Requests.

Benefits:

- Improves code quality.
- Reduces bugs.
- Maintains coding standards.
- Protects important project files.
- Encourages teamwork.

Q20. Examine the relationship between Velocity, Sprint Planning, and Release Planning in Scrum.

In Scrum, **Velocity**, **Sprint Planning**, and **Release Planning** work together to manage project progress.

Velocity is the amount of work completed by a Scrum team during one Sprint. It is measured using Story Points.

Sprint Planning uses the team's Velocity to decide how much work can be completed in the next Sprint.

Release Planning uses the average Velocity of several Sprints to estimate when the complete product will be ready for release.

Relationship:

- Velocity measures team performance.
- Sprint Planning uses Velocity to plan Sprint tasks.
- Release Planning uses Velocity to estimate delivery dates.

Benefits:

- Better planning.
- Realistic workload.
- Predictable project delivery.
- Improved productivity.

Q21. Evaluate whether Scrum or Kanban is more suitable for maintaining an e-commerce application. Justify your answer.

For maintaining an **e-commerce application**, **Kanban** is generally more suitable than Scrum because maintenance work involves continuous bug fixing, feature updates, and customer support. Kanban allows teams to handle tasks as they arrive without waiting for the next Sprint.

Scrum is useful for developing new features in fixed Sprints, while Kanban provides greater flexibility for ongoing maintenance work.

Advantages of Kanban:

- Handles continuous work efficiently.
- Easy to track tasks using a Kanban board.
- Faster bug fixing and issue resolution.
- Improves workflow and productivity.

Example:

If a payment bug is reported by customers, the Kanban team can immediately start working on it without waiting for a new Sprint.

Q22. An organization performs manual testing after development. Evaluate the benefits and challenges of introducing Continuous Integration.

Continuous Integration (CI) automatically builds and tests software whenever developers make code changes. Replacing only manual testing with CI improves software quality and development speed.

Benefits:

- Detects bugs early.
- Reduces manual testing effort.
- Improves code quality.
- Saves time and cost.

- Speeds up software delivery.

Challenges:

- Initial setup requires time and resources.
- Team members need proper training.
- Automation tools may require maintenance.
- Existing workflows may need changes.

Q23. Evaluate different branching strategies (Git Flow, Feature Branching, Trunk-Based Development) for a large-scale software project.

Different branching strategies are used depending on the project size and development process.

Git Flow

- Best for large projects.
- Uses feature, develop, release, and main branches.
- Suitable for planned releases.

Feature Branching

- Each new feature is developed in its own branch.
- Easy to review and test.
- Reduces conflicts.

Trunk-Based Development

- Developers frequently merge changes into the main branch.
- Supports Continuous Integration.
- Best for fast development teams.

Q24. Assess the effectiveness of Retrospectives in improving Agile team performance.

A **Sprint Retrospective** is a Scrum meeting held after every Sprint. During this meeting, the team discusses what went well, what problems occurred, and how to improve in the next Sprint.

Benefits of Retrospectives:

- Improves teamwork.
- Identifies project problems.
- Encourages open communication.
- Increases productivity.
- Supports continuous improvement.

By discussing both successes and challenges, the team can improve its working process and avoid repeating the same mistakes.

Q25. Evaluate the importance of Artifact Repositories such as Nexus in modern CI/CD pipelines.

An **Artifact Repository** is a storage system used to manage software packages, libraries, and build files. **Nexus Repository** is a popular artifact repository used in DevOps and CI/CD pipelines.

Importance of Nexus:

- Stores build artifacts safely.
- Makes software dependencies easy to manage.
- Supports faster deployments.
- Improves security and version management.
- Integrates with CI/CD tools like Jenkins.

Benefits:

- Centralized storage.
- Easy artifact sharing.
- Better software reliability.
- Faster release process.

Q26. Design an Agile development lifecycle for a Library Management System using Scrum.

A **Library Management System** can be developed using the **Scrum framework**, where the project is divided into small Sprints. Each Sprint delivers a working feature of the system. The Product Owner defines the requirements, the Scrum Master manages the process, and the Development Team builds the software.

Scrum Lifecycle:

1. Create the Product Backlog.
2. Plan the Sprint.
3. Develop features such as Login, Book Search, Book Issue, and Return.
4. Test the completed features.
5. Conduct Sprint Review and Retrospective.
6. Repeat the process until the project is completed.

Benefits:

- Faster development.
- Better teamwork.

- Continuous customer feedback.
- Easy to handle changes.

Q27. Create a Git workflow for a team of ten developers that includes feature branches, pull requests, protected branches, and release branches.

For a team of **ten developers**, Git workflow helps manage code safely and efficiently.

Git Workflow:

1. Create a **main branch** for stable code.
2. Each developer creates a separate **feature branch**.
3. Develop and test new features.
4. Push code and create a **Pull Request**.
5. Team members review the code.
6. Merge approved code into the **develop branch**.
7. Create a **release branch** before deployment.
8. Protect the **main branch** so direct changes are not allowed.
9. Merge the release branch into the main branch after final testing.

Benefits:

- Better collaboration.
- Safe code management.
- Fewer merge conflicts.
- Improved software quality.

Q28. Design a Jenkins or GitHub Actions CI pipeline that builds, tests, and packages a Spring Boot application.

A **CI pipeline** automates the process of building, testing, and packaging a Spring Boot application whenever code is pushed to the repository.

CI Pipeline Steps:

1. Developer pushes code to GitHub.
2. GitHub Actions or Jenkins starts automatically.
3. Download project dependencies.
4. Build the Spring Boot application.
5. Run unit tests.
6. Package the application into a JAR file.
7. Store the build artifact for deployment.

Benefits:

- Automatic testing.
- Early bug detection.
- Faster development.
- Reliable software delivery.

Q29. Develop a release planning strategy for a mobile banking application using Agile principles and velocity estimation.

A **release planning strategy** helps organize and deliver features for a mobile banking application. The Product Backlog is prioritized based on customer needs, and work is divided into Sprints. The team's **Velocity** is used to estimate how much work can be completed in each Sprint.

Release Planning Steps:

1. Create and prioritize the Product Backlog.
2. Estimate Story Points for each feature.
3. Plan Sprint tasks based on team Velocity.
4. Review completed work after every Sprint.
5. Release tested features to users.
6. Collect customer feedback and improve future releases.

Benefits:

- Better planning.
- Predictable releases.
- Faster delivery.
- Higher customer satisfaction.

Q30. Propose a complete DevOps adoption roadmap for a software company transitioning from a traditional development process.

A **DevOps adoption roadmap** helps an organization move from traditional software development to a modern DevOps approach.

Roadmap:

1. Assess the current development process.
2. Train employees in Agile and DevOps practices.
3. Adopt Version Control using Git.
4. Implement Continuous Integration (CI).
5. Automate testing and deployment.
6. Use monitoring and feedback tools.
7. Continuously improve the development process.

Benefits:

- Faster software delivery.
- Better collaboration.
- Improved software quality.
- Reduced manual work.
- Higher customer satisfaction.